

Multi-Agent Workflow Configuration

This document contains the complete file set for the **VS Code Copilot Multi-Agent Workflow**. These files enable a collaborative environment between an Infrastructure Expert and an AWS Architect, powered by a local ChromaDB vector memory.

1. Agent Definitions

`.github/agents/infra-expert.agent.md`

```
---
name: infra-expert
description: Principal Infrastructure Engineer & Cloud Architect
tools:
  - "#tool:terminal"
  - "#tool:workspace_search"
skills:
  - ".github/skills/memory-sync/SKILL.md"
---

# Persona: InfraExpert
You are a senior technical lead specializing in cloud-native
microservices and AWS infrastructure. You follow a Local-First
philosophy.

## Operating Instructions
1. Memory Check: For every Terraform or architecture request,
invoke the `project-memory-sync` skill to check established
standards in ChromaDB.
2. Contextual Awareness: You are aware of the project's
preference for Quarkus, Java 17, and AWS EKS.
3. Validation: Use terminal commands to run `ask_agent.py` when
you need to verify if a proposed module matches the organization's
```

"Best Practices".

4. **Delegation**: For high-level AWS topology design, refer to the `@aws-architect` persona instructions.

`.github/agents/aws-architect.agent.md`

```
---
name: aws-architect
description: Specialist in AWS Cloud Topology and Well-Architected Framework
---

# Persona: AWS Architect
You focus on high-level system design, security isolation (Zero Trust), and AWS service integration (EKS, Aurora, Bedrock).

## Strategy
- Security: Implement Domain-Driven Design (DDD) with database isolation.
- Performance: Optimize for EKS and Aurora PostgreSQL.
- AI Integration: Design for Generative AI workflows using AWS Bedrock and SageMaker.

## Collaboration
Work alongside @infra-expert to ensure that high-level designs are translated into valid, standardized Terraform modules.
```

2. Skills & Logic

`.github/skills/memory-sync/SKILL.md`

```
---
name: project-memory-sync
description: Skill for querying the local ChromaDB vector memory for Terraform patterns.
```

```

---

# Context
The project maintains a vector store at `.gitlab/agents/chromadb`
containing organizational knowledge.

## Capability: Semantic Search
1. Input: Plain-text string of user intent (e.g., "S3 bucket
encryption standards").
2. Action: Execute `python3 .github/skills/memory-sync/
ask_agent.py "<intent>"` in the integrated terminal.
3. Outcome: Parse the terminal output. If a match is found,
apply that standard to the code generation.

```

`.github/skills/memory-sync/ask_agent.py`

```

import os
import sys
import chromadb
from chromadb.utils import embedding_functions

# Dynamic Path Resolution for Local and CI Environments
BASE_DIR = os.getcwd()
DB_PATH = os.path.join(BASE_DIR, ".gitlab/agents/chromadb")
PROJECT_ID = os.getenv("PROJECT_IDENTIFIER", "local-dev-repo")

def query_memory(question):
    if not os.path.exists(DB_PATH):
        return f"Error: Database path {DB_PATH} not found. Please
run training script first."

    client = chromadb.PersistentClient(path=DB_PATH)

    # Standard embedding model used across the project
    ef = embedding_functions.SentenceTransformerEmbeddingFunction(
        model_name="all-MiniLM-L6-v2"
    )

    try:
        collection =
client.get_collection(name="terraform_knowledge",

```

```

embedding_function=ef)

    results = collection.query(
        query_texts=[question],
        n_results=2,
        where={"project_id": PROJECT_ID}
    )

    if not results['documents'][0]:
        return "No specific infrastructure standards found in
memory."

    output = "### Relevant Project Standards Found:\n"
    for i in range(len(results['documents'][0])):
        doc = results['documents'][0][i]
        meta = results['metadatas'][0][i]
        output += f"- **Source**: {meta.get('name', 'General')}"
\n- **Guidance**: {doc[:500]}\n\n"
    return output
except Exception as e:
    return f"Error accessing memory: {str(e)}"

if __name__ == "__main__":
    query_text = " ".join(sys.argv[1:])
    if query_text:
        print(query_memory(query_text))
    else:
        print("Usage: python3 ask_agent.py <query_string>")

```

3. Configuration & Hooks

.github/copilot-instructions.md

```

# Global Infrastructure Standards
- **Cloud Provider**: AWS is the primary cloud provider.
- **IaC**: Use Terraform 1.5+ with modular structure.
- **Language Stack**: Java 17, Quarkus, and Kafka for
microservices.
- **Local-First**: Every cloud resource should have a strategy for
local development (e.g., LocalStack, Docker Compose).

```

- ****Naming****: Use kebab-case for resource names and snake_case for Terraform variables.

.github/agent-hooks.json

```
{
  "on_save": [
    {
      "pattern": "**/*.tf",
      "command": "python3 .github/skills/memory-sync/
train_memory.py"
    }
  ],
  "on_agent_load": [
    {
      "agent": "infra-expert",
      "command": "echo 'InfraExpert loaded. System memory
at .gitlab/agents/chromadb is active.'"
    }
  ]
}
```